Smart Lost & Found Management System

Source Code Architecture & C++ Implementation Reference

1. Data Layer

Encapsulates item details including ID, category, location, and metadata via the `Item` class.

2. Logic Layer

Manages item storage and automated matching using string search algorithms in `LostAndFoundSystem`.

3. CLI Interface

Interactive terminal menu for reporting, listing, and triggering the matching engine.

SECTION 1: DATA MODEL DEFINITION

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

using namespace std;

// Class representing an Item (Lost or Found)
class Item {
private:
    int id;
    string name;
    string category;
    string location;
    string date;
    string type; // "LOST" or "FOUND"
    string contact;

public:
    // Constructor
    Item(int id, string name, string category, string location, string date, string type, string
contact) {
        this->id = id;
        this->name = name;
        this->category = category;
        this->location = location;
        this->date = date;
        this->type = type;
        this->contact = contact;
    }

    // Getters
    int getId() const { return id; }
    string getName() const { return name; }
    string getCategory() const { return category; }
    string getLocation() const { return location; }
    string getDate() const { return date; }
    string getType() const { return type; }
    string getContact() const { return contact; }

    // Display item details
    void display() const {
        cout << "----------------------------------------\n";
        cout << "ID: " << id << " [" << type << "]\n";
        cout << "Item Name: " << name << "\n";
        cout << "Category: " << category << "\n";
        cout << "Location: " << location << "\n";
        cout << "Date: " << date << "\n";
        cout << "Contact: " << contact << "\n";
```

SECTION 2: SYSTEM CONTROLLER & ENTRY POINT

```
// Class representing the Lost and Found Management System
class LostAndFoundSystem {
private:
    vector<Item> items;
    int nextId;

    string toLower(string str) const {
        transform(str.begin(), str.end(), str.begin(), ::tolower);
        return str;
    }

public:
    LostAndFoundSystem() { nextId = 1; }

    void reportItem() {
        string name, category, location, date, type, contact;
        int choice;

        cout << "\n--- Report an Item ---\n1. Report Lost Item\n2. Report Found Item\nEnter choice
(1-2): ";
        cin >> choice;
        cin.ignore();

        if (choice == 1) type = "LOST";
        else if (choice == 2) type = "FOUND";
        else { cout << "Invalid choice! Returning to main menu.\n"; return; }

        cout << "Enter Item Name (e.g., iPhone 13, Blue Wallet): "; getline(cin, name);
        cout << "Enter Category (e.g., Electronics, Accessories): "; getline(cin, category);
        cout << "Enter Location: "; getline(cin, location);
        cout << "Enter Date (DD/MM/YYYY): "; getline(cin, date);
        cout << "Enter Contact Info (Email/Phone): "; getline(cin, contact);

        items.push_back(Item(nextId++, name, category, location, date, type, contact));
        cout << "\n[Success] Item registered successfully with ID: " << (nextId - 1) << "\n";
    }

    void displayAllItems() const {
        if (items.empty()) { cout << "\nNo items registered in the system yet.\n"; return; }
        cout << "\n========== ALL REGISTERED ITEMS ==========\n";
        for (const auto& item : items) item.display();
    }

    void findMatches() const {
        if (items.empty()) { cout << "\nNo items available to match.\n"; return; }
        cout << "\n========== SMART POTENTIAL MATCHES ==========\n";
        bool matchFound = false;

        for (size_t i = 0; i < items.size(); ++i) {
            if (items[i].getType() != "LOST") continue;
            for (size_t j = 0; j < items.size(); ++j) {
                if (items[j].getType() != "FOUND") continue;

                bool sameCategory = (toLower(items[i].getCategory()) ==
toLower(items[j].getCategory()));
                bool sameLocation = (toLower(items[i].getLocation()) ==
toLower(items[j].getLocation()));
                bool nameMatch = (toLower(items[i].getName()).find(toLower(items[j].getName())) !=
string::npos ||
                                  toLower(items[j].getName()).find(toLower(items[i].getName())) !=
string::npos);

                if (sameCategory && (nameMatch || sameLocation)) {
                    matchFound = true;
                    cout << "\nPotential Match Found!\nLOST ITEM:\n"; items[i].display();
```